ЗВІТ з практичної роботи № 15

Студент: Anton Romaniuk

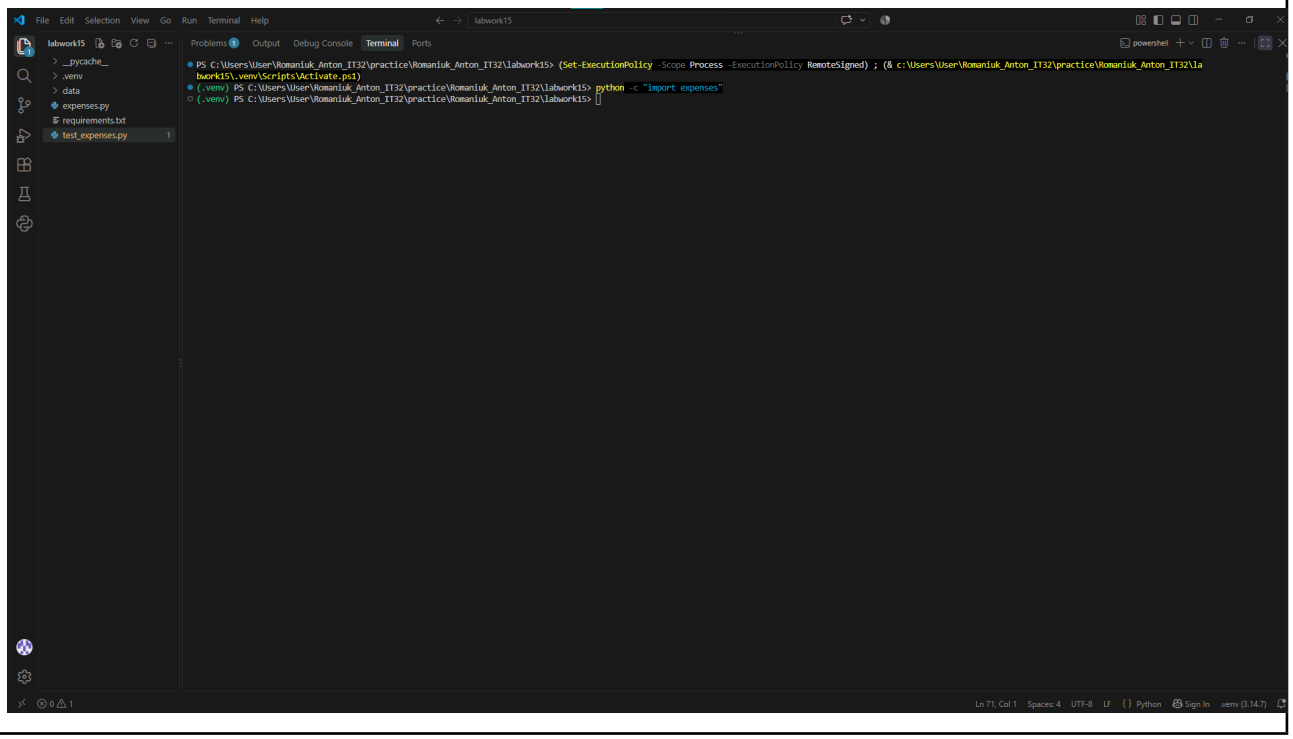
Група: IT-32

Завдання 1. Модуль з функціями

Файл expenses.py містить три функції без print — лише повернення результату:

- parse_line(line) — рядок «2026-09-21;food;85.50;lunch» перетворює на словник {"date", "category", "amount", "comment"}; для неправильної кількості полів, некоректної дати або нечислової суми генерує ValueError;
- total_by_category(records) — словник {category: total}, суми округлені до двох знаків;
- load_expenses(path) — читає файл, пропускає коментарі (#) та порожні рядки, повертає список словників.

```
python -c "import expenses"
```

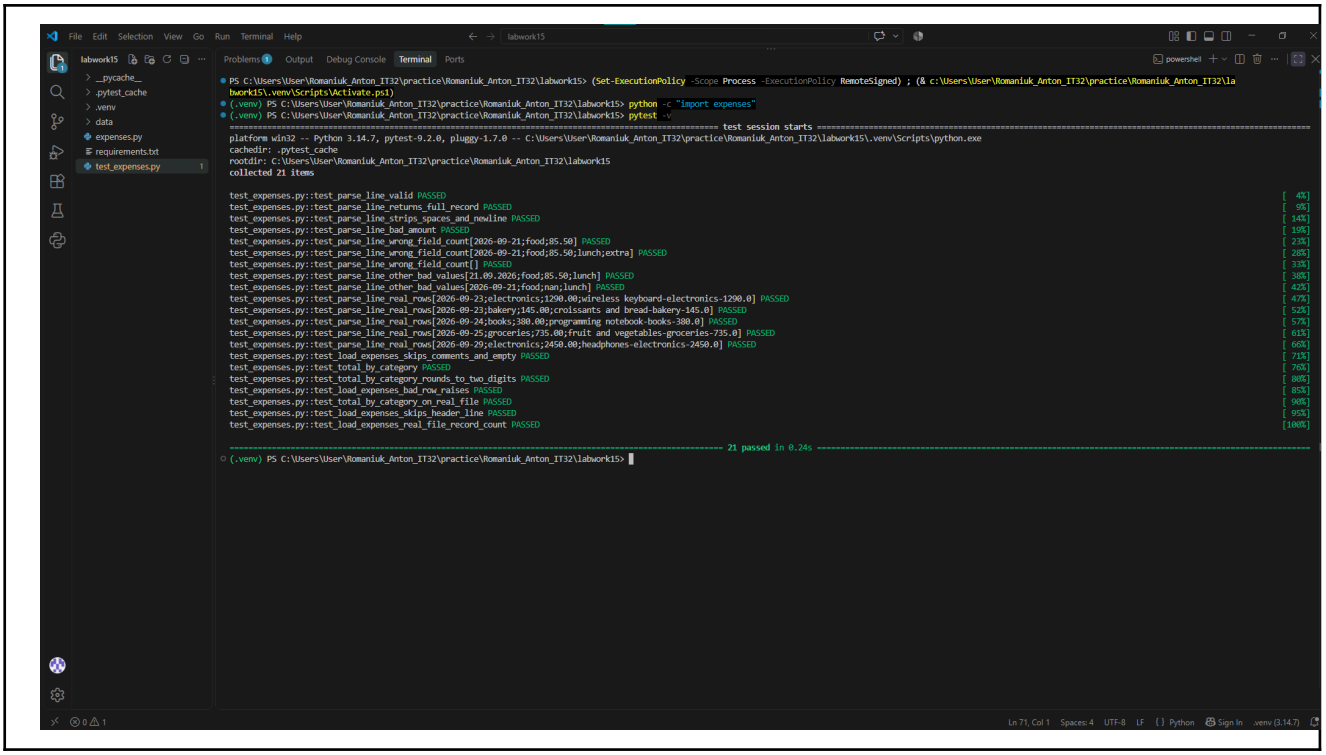


Завдання 2. Перші тести з assert

У файлі test_expenses.py написано тести для parse_line: коректний рядок (категорія й сума), повний словник результату та обробка зайвих пробілів і символу нового рядка.

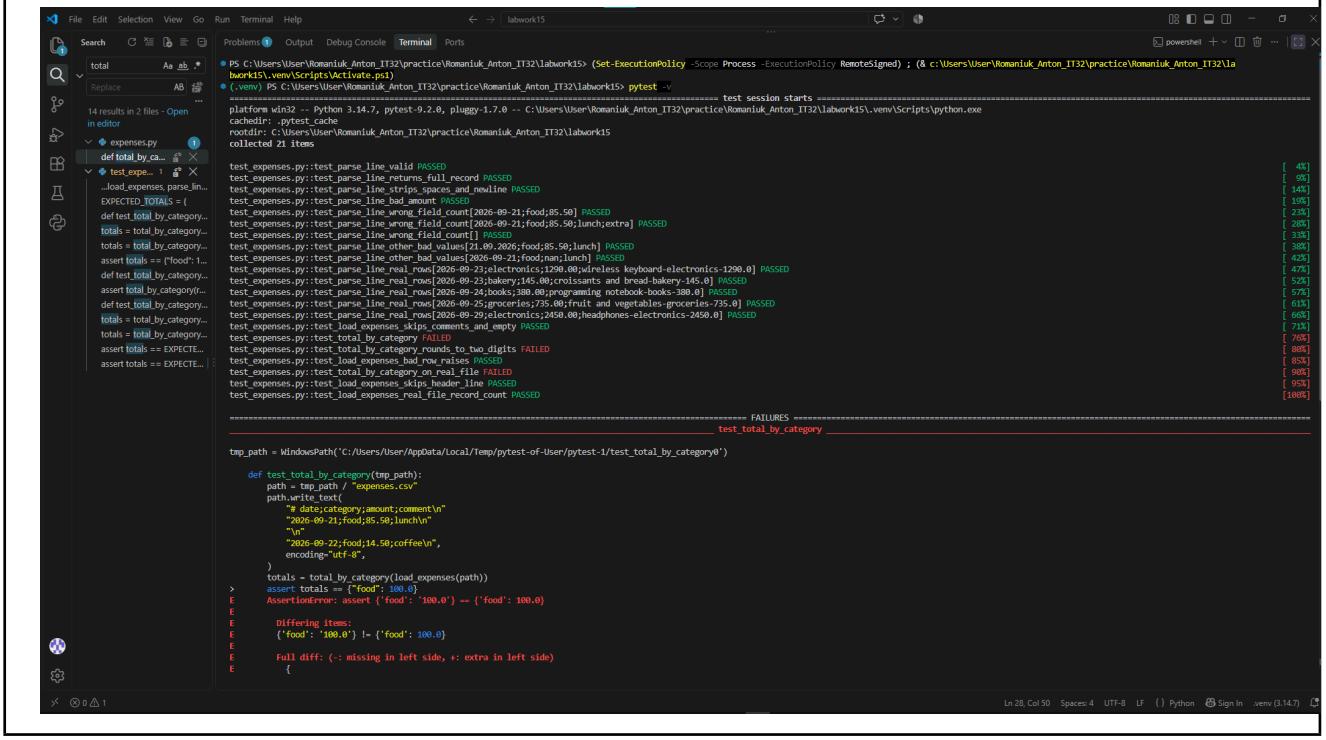
Запуск 1. Усі тести зелені:

```
pytest -v
```



Запуск 2. Навмисна поломка: у функції total_by_category повернути суму як рядок: замінити round(math.fsum(values), 2) на str(round(math.fsum(values), 2)).

pytest -v

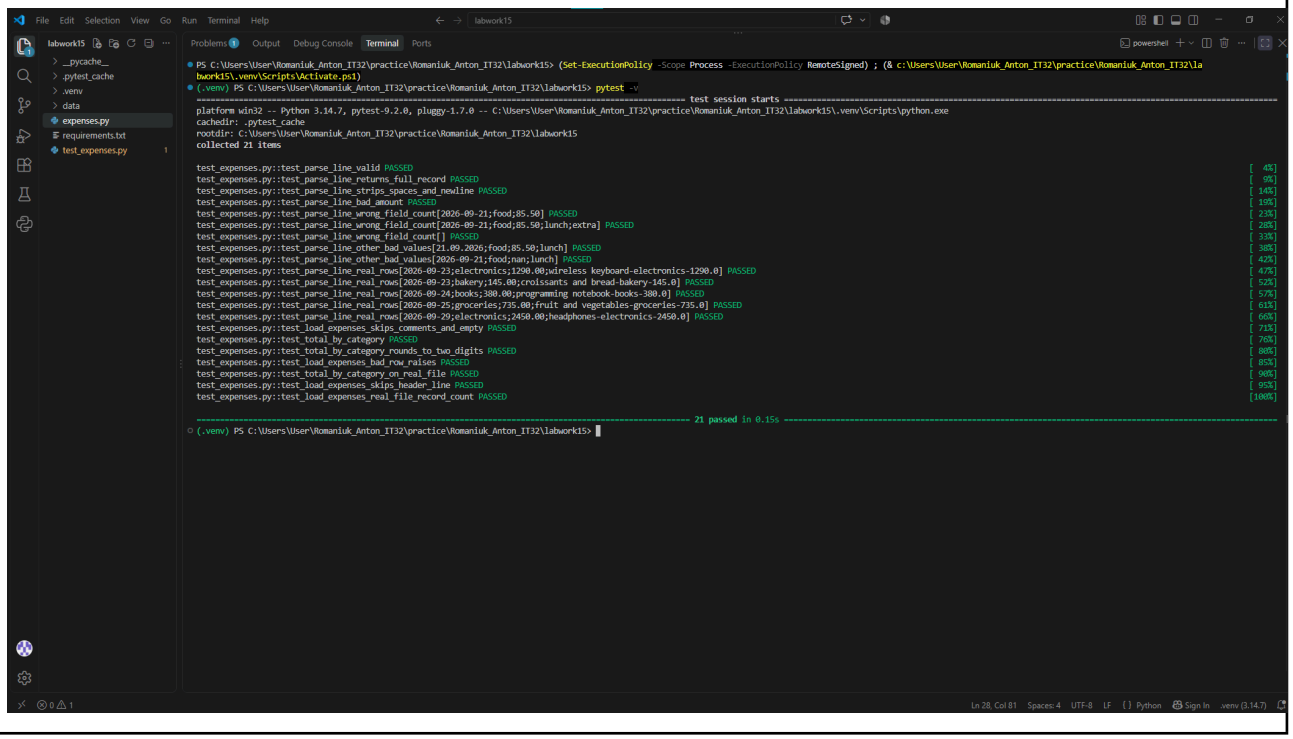


Завдання 3. Перевірка винятків

Тести з pytest.raises(ValueError) перевіряють два різні види помилок: **нечислова сума** (2026-09-21;food;abc;lunch) і **неправильна кількість полів** (замало, забагато, порожній рядок); додатково — неправильний формат дати та значення nan.

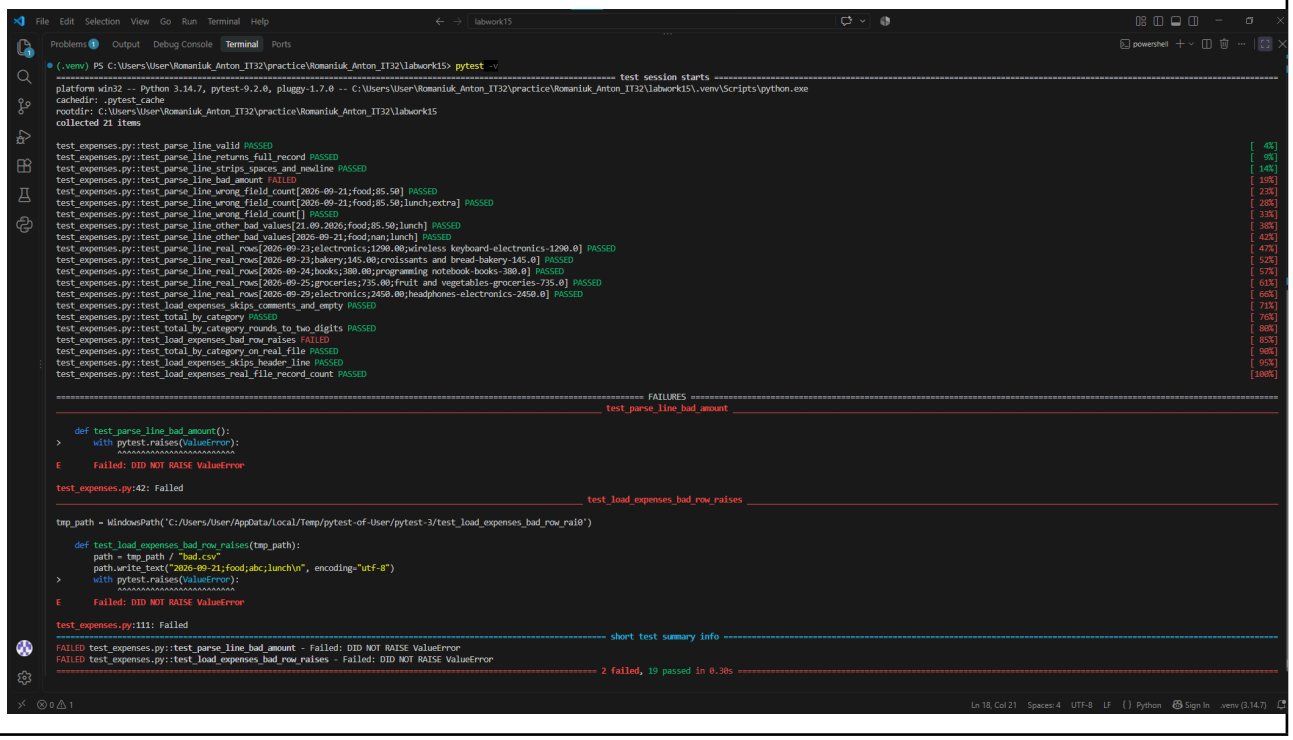
Запуск 1. Тести проходять:

pytest -v



Запуск 2. Прибрано генерацію винятку: у parse_line замість raise ValueError(f"bad amount: ...") стоїть amount = 0.0.

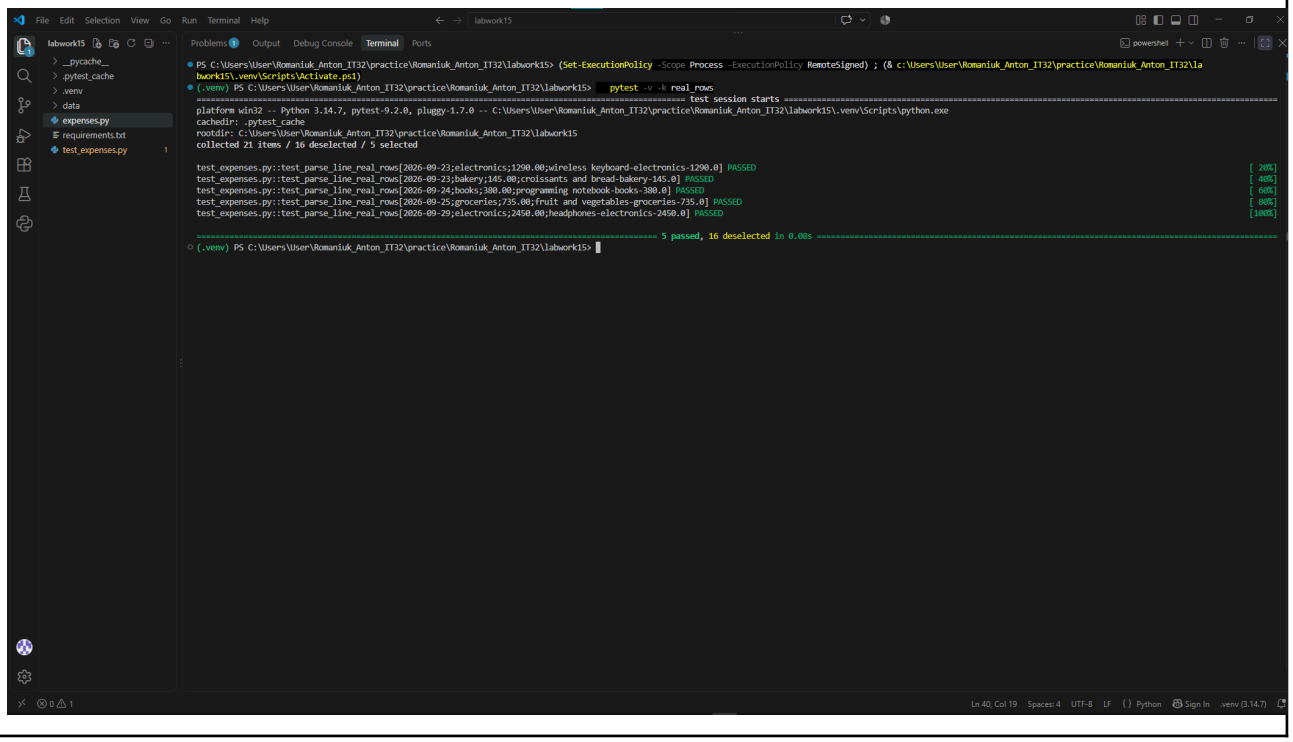
pytest -v



Завдання 4. Параметризований тест

За допомогою @pytest.mark.parametrize функція parse_line перевіряється на 5 рядках з мого файлу витрат.

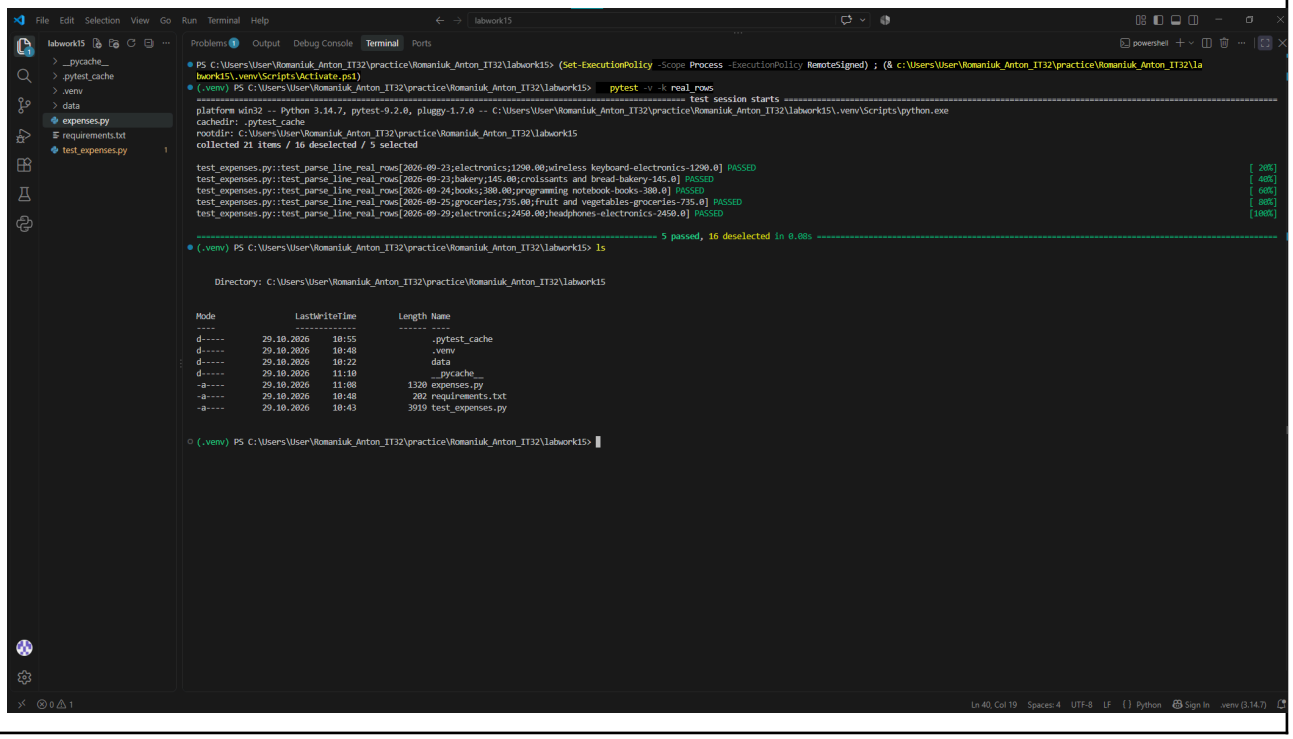
pytest -v -k real_rows



7. Завдання 5. Тест з тимчасовим файлом

Тест test_total_by_category створює файл у каталозі, наданому фікстурою tmp_path, записує туди коментар, рядки витрат і порожній рядок та перевіряє результат total_by_category(load_expenses(path)) (очікується {"food": 100.0}). Ще один тест перевіряє total_by_category на справжньому data/expenses.csv; очікувані суми взято з результату роботи 22.

ls



Відповіді на контрольні запитання

1. Що таке unit-тест і чим він відрізняється від ручної перевірки програми? Unit-тест — це невелика автоматична програма, яка викликає одну функцію (одиницю коду) з відомими вхідними даними й перевіряє, що результат збігається з очікуваним. Тест можна запускати будь-коли однією командою (pytest), він швидкий і повторюваний, фіксує, як має поводитися функція, і одразу виявляє, що після зміни коду щось зламалося (регресію).

- ручна перевірка потребує щоразу запускати програму, вводити дані й дивитися на результат очима: це повільно, залежить від уважності людини, легко пропустити граничні випадки або просто забути перевірити;
- unit-тест перевіряє кожну функцію окремо й однаково щоразу, тож місце помилки знаходиться швидко.

2. Як pytest знаходить тести в каталозі проєкту?

Запущений без аргументів, pytest рекурсивно обходить поточний каталог (і його підкаталоги) та збирає файли з назвами test_*.py або *_test.py. У цих файлах тестами вважаються функції, назви яких починаються з test_ (а також методи класів Test*). Тому файл з тестами я назвав test_expenses.py, а функції — test_parse_line_valid тощо. Функції без префікса test_ pytest не запускає.

3. Чому тест обов'язково треба побачити таким, що падає?

Тест, який жодного разу не падав, може взагалі нічого не перевіряти: помилитися в очікуваному значенні, перевіряти не ту функцію або мати умову, що завжди істинна. Тільки коли після навмисної поломки коду тест червоніє (AssertionError або DID NOT RAISE), ми переконуємося, що він справді здатний знайти помилку. Тому в роботі я спочатку отримував зелені тести, а потім ламав функцію й дивився, що тест її помітив.

4. Як перевірити, що функція генерує виняток?

Використовують контекстний менеджер pytest.raises:

- with pytest.raises(ValueError): parse_line("2026-09-21;food;abc;lunch"); • якщо всередині блоку виняток не виникне, тест впаде з повідомленням DID NOT RAISE; якщо виникне виняток іншого типу, тест теж впаде;
- за потреби можна перевірити й текст помилки через параметр match=.

5. Навіщо параметризувати тести замість копіювання однакових функцій?
@pytest.mark.parametrize дозволяє написати тіло тесту один раз і прогнати його на багатьох наборах даних. Це прибирає копіпаст, а додати новий випадок — це додати один рядок у список. Кожен набір pytest запускає як **окремий тест** з параметрами в назві (test_parse_line_real_rows[...]), тож видно, який саме випадок упав, і помилка в одному не перериває перевірку решти, на відміну від кількох assert в одній функції.

6. Чому тест не повинен залежати від файлів, які можуть змінитися, і як у цьому допомагає tmp_path?

Якщо тест читає файл, який можна редагувати, переміщати або якого немає на іншому комп'ютері, результат стає непередбачуваним: сьогодні тест зелений, завтра після зміни даних червоний, хоча код правильний. Такі тести не відтворювані та їм не можна довіряти. Фікстура tmp_path дає кожному тесту власний унікальний тимчасовий каталог (pathlib.Path), у якому тест сам створює файл із відомим вмістом. Тож очікуваний результат завжди передбачуваний, тести ізольовані один від одного, а в каталозі проєкту не лишається зайвих файлів: pytest сам керує цими каталогами.